

Your first background job

Build a small API whose slow work happens in a background job — the endpoint answers instantly, a status endpoint reports progress, and one cron job runs on the clock alone.

~5–6 h across 6 stages

Stretch +2 h

Bonus AI stage +1 h

JavaScript or Python

\$0 · no credit card

PAIRED LIVE EVENT

When the request can't wait —
background jobs & durable
workflows

YOU WILL PRACTICE

Background jobs · Events ·
Status endpoints · Retries · Cron
schedules

SUBMISSION

Public GitHub repo, ≥6 commits,
README

How to read this document: new words are shown in **bold** the first time they appear — every one of them is explained in the [Glossary](#) at the end. Work the stages **in order**; each ends with a checkpoint that proves it works.

Contents

1. Goal & purpose	1	6. Requirements & stretch goals	7
2. The big idea in 60 seconds	2	7. Done means	7
3. Tools — pick one lane	2	8. Curated resources	8
4. The task — six stages	2	9. Glossary	10
5. Bonus stage — the AI rematch	6		

1 · Goal & purpose

Goal: build a small **API** with one slow task — a report that takes 8 seconds to make. The API answers instantly and gives the slow work to a **background job**. A **status endpoint** tells the client when the report is ready. You also build one **cron job**: a task that runs on a clock, with no request at all. You publish everything to **GitHub**.

In the lecture you saw the 40-second handler break: the user waits, the request times out, the user retries, the work happens twice. Now you build the professional fix yourself. The pattern has three parts: **accept fast, work in the background, report status**. Every "we'll email you when it's ready" you have ever seen is this pattern. FlyRank runs on it too — SEO reports and AI images are made by background jobs, not inside requests.

This assignment stands alone. You do not need the code from earlier assignments — but if you have your CRUD API, you may build on it.

Beginners usually overthink this. The whole thing is under 120 lines of code. You never write more than ~15 lines before you can test again. And you do not build the hard machinery (queues, workers, retries) yourself — a free tool called **Inngest** does that part, and shows you every job in a visual **dashboard**.

2 · The big idea in 60 seconds

Until now, all your work happened *inside* a request: the client asks, the server does the work, the client waits. That is fine for fast work. It breaks for slow work. The fix: there are three different ways work can start.

How the work starts	Example	The name
A client asks and waits for the answer	<code>GET /health</code>	Request/response — everything you built so far
A client asks, and the work happens later	"Make my report" → <code>202</code> now, report in 8 s	Background job , started by an event
Nobody asks. The clock starts it.	"Every day at 08:00, send the summary"	Cron job , started by a schedule

That table **is** the assignment. You build one endpoint from row 1, one job from row 2, and one job from row 3.

And what is a cron job, exactly? A cron job is a task that a scheduler starts at fixed times — every minute, every day at 08:00, every Monday. The name comes from `cron`, the Unix scheduler that has done this work since the 1970s. You describe the schedule in a **cron expression**: five short fields that mean "when". Stage 4 teaches you to read and write one.

3 · Tools — pick ONE lane

Both lanes build exactly the same thing. Pick the language you want to stick with; don't switch mid-assignment.

	JavaScript lane	Python lane
Language	Node.js (free, nodejs.org)	Python 3.10+ (free, python.org)
Web framework	Express — you know it from A1	FastAPI — you know it from A1
Background jobs	Inngest — Node.js quick start	Inngest — Python quick start
The job dashboard	Inngest Dev Server — one command, runs on your machine, no account, no card	
Testing	<code>curl</code> + browser + the dashboard	
Publishing	Git + a free GitHub account	

Why Inngest? It is free for this entire assignment, it runs fully on your machine, and it gives you a dashboard where you can *watch* your jobs run, retry, and fail. It is also the tool FlyRank uses in production — you are learning the real thing. Other job tools exist (BullMQ, Celery, node-cron); they are listed in resources §5 so you recognize them later. For this assignment, use Inngest.

4 · The task — six stages (+ one bonus)

Work stage by stage, **in order**. Each stage ends with a checkpoint: a command you run, or a thing you see, that proves it works. Commit to Git after every stage (that's your ≥ 6 commits, honestly earned). If you only finish Stage 3, submit anyway — **a working half is worth more than a broken whole**.

0 Hello, server

~30 min

Every job system starts as a normal API.

1. Create a small server with one endpoint: `GET /health → { "status": "ok" }`. You built this in A1 — a fresh copy of that first stage is all you need here.
2. Start it: Express on **port 3000**, FastAPI on port 8000.

CHECKPOINT — `curl -i http://localhost:3000/health` (or `:8000/health`) returns **status code 200** and the JSON.

Commit: `Stage 0: hello server`

1 Hire the worker: connect Inngest

~1 h

You now hire an assistant who works in a separate room. First, introductions.

1. Install Inngest: `npm install inngest` · `pip install inngest`.
2. Follow your lane's quick start (section 3) to do three small things: create the Inngest **client** (id `report-api`) · create one **function** named `say-hello`, **triggered** by the event `test/hello` — inside it, sleep 5 seconds (`step.sleep`), then return `"Hello from the background!"` · **serve** the function at the path `/api/inngest` (the quick start shows the exact lines for Express and FastAPI).
3. Start the **Dev Server** in a second terminal:

```
npx inngest-cli@latest dev -u http://localhost:3000/api/inngest
```

(use your port).

4. Open the dashboard: `http://localhost:8288`. Find your function and press **Invoke**.

Two programs are now running: your API, and the Inngest Dev Server. Keep both terminals open for the rest of the assignment.

CHECKPOINT — the dashboard shows a run of `say-hello` with the 5-second sleep step, ending **Completed**.

Commit: `Stage 1: Inngest connected, first function runs`

2 The fast door: accept now, work later

~1.5 h

A customer orders a cake. The bakery does not make the customer stand at the counter for 8 seconds — it gives them a receipt with a number.

1. Near the top of your file, create an **in-memory** dictionary/map for reports. (Yes, it forgets everything on restart — same lesson as A1, still not a bug.)
2. Add `POST /reports`. The client sends `{ "topic": "cats" }`. Your endpoint: makes an id and saves `{ "id": ..., "topic": "cats", "status": "pending" }` · **sends the event** `report/requested` with the id and topic in `data (inngest.send(...))` · returns immediately with status **202** ("Accepted" — "I have the order, work starts soon") and the body `{ "id": ..., "status": "pending" }`. The endpoint does **no slow work**. That is the whole point.
3. Create a second Inngest function, `make-report`, triggered by `report/requested`. Inside, use two **steps**: `step.sleep("do-the-slow-work", "8s")` — a stand-in for a real slow task (an AI call, a big export) — then a `step.run("build-report", ...)` that builds the result and saves it into the map with `status: "done"`.
4. Add the status endpoint `GET /reports/:id` → the saved object: first `pending`, later `done` + the result. Unknown id → **404**. A client asking again and again is called **polling** — and "first pending, then done" has a grown-up name: **eventual consistency**.

CHECKPOINT — `time curl -i -X POST http://localhost:3000/reports -H "Content-Type: application/json" -d '{"topic":"cats"}'` returns **202** in **well under one second**. `GET /reports/<id>` says `pending`; the same command ~10 seconds later says `done` with the result.

Stop and notice: **the request is fast even though the work is slow**. You just built the pattern behind every progress bar and every "we'll email you when it's ready" on the internet.

Commit: Stage 2: 202 + background job + status endpoint

3 Jobs fail. Watch the retry.

~45 min

The oven breaks mid-cake. A good bakery tries again — and knows when to stop.

Background jobs fail: networks drop, services hiccup. A job tool's answer is the **retry** — run the failed job again, automatically, after a short wait. Inngest retries failed runs for you; you just need to see it happen.

1. In `make-report`, add one line at the top of the `build-report` step: if the topic is `"fail"`, throw/raise an error (`"The report oven is broken!"`).
2. Set the function's retries to 2 so the show is short: **retries: 2** in the function config · **retries=2** in `create_function`.
3. `POST /reports` with topic `"fail"`. Open the run in the dashboard and watch: attempt 1 fails → wait → attempt 2 → attempt 3 → the run ends **Failed**. The waits get longer each time — that is called **backoff**.
4. Note what did *not* need a retry: a request with a missing topic. Add validation to `POST /reports`: no topic → **400**, and no event is sent. A wrong *input* must be rejected at the door; only a wrong *moment* deserves a retry. Write one sentence about this difference in your README.

CHECKPOINT — the dashboard shows a `make-report` run with 3 attempts and final status **Failed**. `POST /reports` with no topic returns **400** and creates no job.

Commit: Stage 3: retries seen, bad input rejected

4 The clock knocks: your first cron job

~45 min

Nobody asks the sun to rise. Some work must simply happen every morning.

Cron in five fields. A **cron expression** is five values, left to right: **minute** · **hour** · **day-of-month** · **month** · **day-of-week**. A ***** means "every". Examples: `* * * * *` = every minute · `0 8 * * *` = every day at 08:00 · `0 9 * * 1` = every Monday at 09:00 · `*/15 * * * *` = every 15 minutes. You never have to decode these in your head: paste any expression into crontab.guru and it explains it in plain words. Servers usually run cron in UTC — check the timezone before you trust a schedule.

1. Create a third Inngest function, `heartbeat`, with a **cron trigger** instead of an event trigger: schedule `* * * * *` (every minute — only for testing; a real one would run daily). `{ cron: "* * * * *" } . inngest.TriggerCron(cron="* * * * *")` — see the [scheduled functions guide](#).
2. Inside, log one summary line: how many reports are `pending`, `done`, `failed`. No endpoint, no event — the clock is the only trigger.
3. Watch the dashboard for two minutes. Then answer in your README, one sentence each: which cron expression would run this **every day at 08:00**? Which would run it **every Sunday at 22:00**? (Build them on crontab.guru.)

CHECKPOINT — the dashboard shows two `heartbeat` runs, one minute apart, each with your summary line.

Commit: `Stage 4: cron heartbeat`

5 Publish to GitHub

~45 min

Your work only counts when someone else can run it.

1. Put the code in a **public GitHub repo**. Assignments in this program are independent: a new repo is fine, and adding this to a repo you used for another assignment is also fine. If one repo holds several assignments, give each assignment its own clearly named folder (for example `background-job/`) so anyone can find the right work quickly. The public repo link is what you submit.
2. Write a **README** with: what this is, how to run it (**two documented commands** — your API and the Dev Server), a table of the endpoints and the three functions, one pasted proof (the `202` response + the two polls), your Stage 3 and Stage 4 sentences, and a screenshot of the dashboard with your runs.

CHECKPOINT — a stranger with your README could run both terminals and get a report in under 5 minutes.

Commit: `Stage 5: publish and docs` — then push everything.

★ Make it yours — optional extras

optional · as many or as few as you like

The job system is yours now. Give it a personality.

None of these are required. Pick whatever sounds fun (creative alternatives welcome):

- **A list endpoint:** `GET /reports` returns all reports and their statuses — your job system's control panel.
- **The "email":** instead of only saving the result, the job also writes it to a file `outbox/<id>.txt` — a stand-in for **sending mail from a job**, which is where this pattern lives in real products.
- **A cleanup cron:** a second scheduled function that deletes `done` reports older than 10 minutes — cron's most common real job is taking out the trash.
- **Design your own schedule:** change the heartbeat to a schedule you build on crontab.guru — and write in the README what it means.
- **The restart experiment:** start a report, and while the job is inside its 8-second sleep, stop your API (Ctrl-C), count to three, start it again. Watch the dashboard. Write two sentences in your README about what happened to the job. *This observation — the job survived; your server didn't have to — is called being **durable**, and it is the deep magic of tools like Inngest.*

Commit (if you build any): `Extras: <what you added>`

5 · Bonus stage — the AI rematch

6 The AI rematch

~1 h · optional — and the most fun

You built a job system by hand. Now hire the fastest junior developer on Earth — and review their work.

You did Stages 0–5 by hand for a reason: you now know exactly what "correct" looks like. That knowledge turns this stage from a magic show into a code review.

1. **Write the prompt yourself — this is the real exercise.** Without copying text from this document, write your own **prompt** asking an AI assistant (Claude, ChatGPT, Gemini — any) to build the same system. From memory, specify everything that matters: the instant `202 + id`, the status endpoint and its states, the background function with a sleep and a build step, the retry behavior, the `400` for bad input, the every-minute cron. Describing a system precisely is a core backend skill.
2. **Generate in quarantine.** The AI's code goes in a separate folder (`ai-version/`) or branch. Your hand-built submission stays hand-built.
3. **Run it.** Fire your Stage 2 and Stage 3 checkpoints at it. Which pass? Which fail?
4. **Diff it.** Compare side by side (`git diff --no-index` works on any two files). Then answer three questions in an "AI vs me" section of your README: What did the AI do *better* — and do you understand it? What did it get *wrong* or *silently ignore*? What did *your prompt* forget to specify — and what did the AI silently decide for you?
5. **One rematch.** Improve the prompt, regenerate, and note in one sentence what changed.

The lesson hiding in this stage: an AI's output is exactly as good as your specification — and you could only judge it because you built the thing yourself first. **Both halves of that sentence are your career from now on.**

CHECKPOINT — README has an "AI vs me" section with your full prompt and ≥ 3 concrete differences.

Commit: `Stage 6: AI vs me` (AI code stays in its own folder/branch).

6 · Requirements

Done = every box ticked. Each one is checkable in under a minute.

- ☐ The API and the Inngest Dev Server each start with **one documented command**.
- ☐ `POST /reports` returns **202 + an id in under one second**; the ~8-second work happens in a background function, not in the endpoint.
- ☐ `GET /reports/:id` answers **pending** first, **done** + result later; unknown id → **404**; missing topic on POST → **400**.
- ☐ The `make-report` function uses **at least two steps** (a sleep and a run) and its runs are visible in the dashboard.
- ☐ A run with topic **"fail"** shows **3 attempts** and ends **Failed** in the dashboard.
- ☐ A **cron function** runs every minute and logs one summary line.
- ☐ Public **GitHub repo** (own repo, or a clearly named folder in a shared one), ≥ 6 meaningful commits, README with the two run commands, endpoint/function table, the pasted **202** -then-poll proof, your Stage 3 & 4 sentences, and the dashboard screenshot.

Stretch (optional)

- ☐ **Idempotency**: send the *same* `report/requested` event twice with the same id — and make sure the report is built only **once** (a check on the saved status does it). Explain in one README line why jobs must survive running twice.
- ☐ **Concurrency limit**: configure the function to run at most 2 reports at once, enqueue 5, and watch 3 of them wait. One README line: when would you *want* a queue to be slow?
- ☐ **Durable, proven**: do the restart experiment from the extras, but with a 3-step function — and show (dashboard screenshot) that finished steps did **not** run again after the restart.
- ☐ The AI rematch (section 5): prompt an AI to build the same system, run it, diff it, and write your "AI vs me" section.

7 · Done means

- The **202** proof: your README shows the POST answering in milliseconds and the poll flipping from **pending** to **done** seconds later.
- The dashboard screenshot shows all three functions: a completed `make-report` with its steps, a failed run with its retries, and the `heartbeat` cron runs.
- The repo is public, the README works on a clean machine, and `git log` shows one honest commit per stage.

8 · Curated resources

Don't read everything. Each section says *when* you need it. All resources are free, no credit card.

● Start here (everyone) · ● When you're comfortable · ● Optional deep dive · JS / Py = lane-specific

§1 · The idea — why work leaves the request

Resource	Format	Why it's useful
● Keeping your API fast (Inngest)	Article, ~5 min	The whole assignment in one page: move slow work out, return early, report status. Read before Stage 2.
● Worker dynos, background jobs and queueing (Heroku)	Article, ~10 min	The classic explanation of <i>why</i> web requests must stay fast, with good pictures.
● Background jobs — setup guide (Inngest)	Docs, ~10 min	The event-driven view of jobs, plus fan-out. Read after Stage 2 to see where the pattern goes next.

§2 · Official docs — your lane's exact pages

Resource	Format	Why it's useful
● JS Inngest Node.js quick start	Tutorial, ~20 min	The exact wiring for Stage 1: client, function, serve, dev server. Keep open during Stages 1–2.
● Py Inngest Python quick start	Tutorial, ~20 min	The same wiring for FastAPI: <code>pip install inngest</code> , <code>inngest.fast_api.serve(...)</code> .
● Local development — the Dev Server	Docs, ~5 min	What the dashboard at <code>localhost:8288</code> is and can do. Read when a run "disappears".
● Inngest functions — durable background logic	Docs, ~15 min	Triggers, steps, retries in one place — the map of everything Stages 2–4 touch.
● Py Python create_function reference	Reference	The exact <code>retries=</code> and <code>TriggerCron</code> syntax for Stages 3–4. Look up, don't read.
● JS Serving Inngest functions	Docs, ~10 min	If your framework wiring differs from the quick start — the serve recipe per framework.

§3 · Cron — reading and writing schedules

Resource	Format	Why it's useful
● crontab.guru	Interactive tool	Paste any cron expression, get plain words back. Keep open for all of Stage 4.
● crontab.guru — examples	Reference	The common schedules ("every day at 8", "every Monday") ready to copy.
● Scheduled functions & cron jobs (Inngest)	Docs, ~5 min	The cron trigger you use in Stage 4, including the <code>TZ=</code> timezone prefix.
● A beginner's guide to cron jobs (OSTechNix)	Article, ~15 min	Where cron comes from: the classic Unix crontab, field by field. Read after Stage 4.

● JS	Schedule a job in Node with node-cron (freeCodeCamp)	Tutorial, ~10 min	The minimal do-it-yourself alternative: cron inside your own process — good contrast to what Inngest gives you.
●	Cron jobs: the complete guide (Cronitor)	Article, ~20 min	Everything else: special strings, pitfalls, monitoring cron in production.

§4 · Build-alongs — type along, then build your own

Watching or typing along is allowed and encouraged. The final submission must be your own — your endpoints, your report, your sentences.

	Resource	Format	Why it's useful
● JS	No workers necessary — background jobs with Node and Express	Article, ~30 min	Nearly this exact assignment: an Express endpoint sends an event, a function does slow work. Do it before Stage 2 if you want training wheels.
● Py	The Python quick start, typed out fully	Tutorial, ~30 min	The Python lane's best build-along is the official one — do it as written first, then adapt it into Stage 2.
●	Next.js background jobs / cron / queues with Inngest	Video, ~30 min	Same ideas in a Next.js app — the framework differs, the concepts (events, steps, cron) transfer 1:1.

§5 · Other tools you will meet (recognition only — not for this assignment)

	Resource	Format	Why it's useful
● JS	BullMQ introduction	Docs	The classic Node queue on Redis: you manage the queue and the worker yourself. What Inngest automates, spelled out.
● Py	First steps with Celery	Tutorial	Python's veteran job system — broker, worker, tasks. You will see it in many jobs and codebases.

Critical resources per stage

Stage	Keep these open
Stages 1–2	Your lane's quick start (§2) · the Dev Server page · the build-along (§4) if you want to type along first.
Stage 3	The retries rows in §2 — and the lecture's resilience-kit segment, worth a rewatch.
Stage 4	crontab.guru in a tab · the scheduled functions guide · §3 for a deeper cron read.
Stage 5	The Git & GitHub guide in the program resources — init → add → commit → push is all you need.

Suggested path for a complete beginner (~1 h before starting)

1. [Keeping your API fast](#) — ~5 min. The idea.
2. Your lane's quick start (§2) — ~20 min, typed along. The wiring.
3. [crontab.guru](#) — 5 min of playing: build "every day at 08:00". The clock.
4. Start Stage 0. Come back to §3–§4 only when a stage sends you here.

Everything above is free. If a link ever dies, the titles are searchable — the material outlives the URL.

9 · Glossary

Plain-language definitions of every **bold** word above. No definition depends on another — read them in any order.

Word	What it means
API	A set of doors a program offers so other programs can talk to it — here, your report service.
Background job	Work your server starts but does not finish inside the request. The client gets an answer now; the work finishes later, elsewhere.
Worker	The program (or part of a program) that actually does background work. In this assignment, Inngest plays the worker's role for you.
Event	A small message that says "something happened" — like <code>report/requested</code> . Functions can be set up to run whenever a matching event arrives.
Trigger	The thing that starts a function: an event ("when a report is requested") or a schedule ("every minute").
Status endpoint	An endpoint whose only job is to report progress: <code>GET /reports/:id</code> → <code>pending</code> , <code>done</code> , or <code>failed</code> .
Polling	Asking the status endpoint again and again until the answer changes. Simple, and completely normal.
202 Accepted	The status code that means: "I received your request and work will happen — but it is not done yet." The honest answer of every fast door.
Status code	The 3-digit number in every response saying how it went: <code>200</code> OK, <code>202</code> Accepted, <code>400</code> Bad Request, <code>404</code> Not Found.
Eventual consistency	The state of the world between "accepted" and "done": the answer exists <i>soon</i> , not <i>now</i> . UIs handle it by showing progress.
In-memory	Data kept in your program's variables. Fast and simple — and gone when the program stops. (Databases fix this; you met that lesson in A1/A2.)
Step	One named piece of a job, like <code>do-the-slow-work</code> or <code>build-report</code> . Job tools save each finished step, so a re-run can skip what is already done.
Retry	Running a failed job again automatically. Good for temporary failures (network hiccup), wrong for bad input (missing topic).
Backoff	Waiting longer before each retry — 5 s, then 30 s, then 2 min — so a struggling service gets air instead of more punches.
Cron job	A task started by a scheduler at fixed times — every minute, every day at 08:00 — with no request involved.
Cron expression	The five-field code that describes a schedule: minute, hour, day-of-month, month, day-of-week. <code>0 8 * * *</code> = every day at 08:00.
Schedule	The "when" of a cron job — written as a cron expression.
Inngest	The free tool this assignment uses to run background jobs and cron jobs. Your code defines the functions; Inngest triggers them, retries them, and shows them in a dashboard.
Dev Server	Inngest's local engine + dashboard, started with one command. It runs only on your machine and needs no account.

Dashboard	The Dev Server's web page at <code>localhost:8288</code> : every function, every run, every step, every retry — visible.
Client	Whatever sends the request: a browser, curl, a mobile app.
Port	A numbered doorway on a machine, so many programs can listen at once — your API on 3000/8000, the dashboard on 8288.
Validate / validation	Checking incoming data before trusting it (is <code>topic</code> there?). The server never assumes the client behaved.
Idempotency	The property that running the same job twice causes the effect once . Jobs <i>will</i> run twice someday; idempotent jobs don't care.
Concurrency limit	A cap on how many jobs run at the same time (e.g. 2). The rest wait in line. Protects slow services from being trampled.
Durable	Work that survives crashes and restarts: because each step's result is saved, a restarted job resumes where it stopped instead of starting over.
Git / GitHub / repo / commit	Git tracks versions of your code in a repository ; a commit is one saved step with a message; GitHub hosts your repo online so others can see it.
README	The front page of a repo: what this is, how to run it, how to use it.
Prompt	The instructions you give an AI assistant. In the bonus stage your prompt is a mini-specification — the more precisely it names behaviors, the closer the output lands.
Diff	A line-by-line comparison of two versions of code. <code>git diff</code> produces one; reading diffs is how professionals review each other's work.

FlyRank Internship · Backend Development Track · Week 4 · Assignment A7 — Your first background job. All linked resources are free with no credit card required.